Consistency for size() functions

Document #:	P1970R1
Date:	2020-01-13
Project:	Programming Language C++ Library Working Group

Reply-to:	Hannes Hauswedell <h2@fsfe.org>

Contents

§ 1 History

§ 1.1 R1

Clean-up of wording and integration of wording fixes based on new discussion points. The paper body is unchanged, but new discussion points have been added as “Discussion (after R0)”.

It is my understanding that these changes are in LWG’s realm, but I will double-check with LEWG-chair to see if he wants to see this again.

While I think that this paper is important and *should* make it into C++20, the urgency and implications for breakage are significantly reduced by the new wording, because the behaviour of `std::size()` and `std::ssize()` is largely preserved for arguments that are not `sized_ranges`.

1.2 R0

Created during Belfast (Fall 2019) in response to DE269.

2 Introduction

C++17 introduced `std::size()` and C++20 will introduce `std::ssize()` and `std::ranges::size()`. Why do we need three and do they solve all problems in regard to size? After DE269 raised these questions, LEWG requested I write this paper.

3 Motivation and Scope

	`.size()`	`.size()` or `end() - begin()`
unsigned †	`std::size()`	`std::ranges::size()`
signed	`std::ssize()`	*none*

† *This can be signed under certain circumstances, but is not for any types in the standard library. Criticism of this was raised in LEWG, but this is independent of everything else and not touched by this paper to reduce last-minute impact.*

`std::ranges::size()` was introduced (also) because it was desired that subtractable iterator-sentinel-pairs should be sufficient for a range to qualify it as a `std::ranges::sized_range`. This means `std::size()` will not work on certain sized ranges, but `std::ranges::size()` will.

`std::ssize()` has been introduced (with some controversy), because there was the desire to have a signed size type that can be used (among other things) in loops and comparisons with signed counter variables. It is defined as `std::size()` but casts the type to a comparable signed type. This solves the problem but only works on the subset of sized ranges that `std::size()` works on.

3.1 There is no ssize that works for all ranges

The first problem and original intent of DE269 is that there is no signed size function that works on all sized ranges, i.e. if the arguments for adding `std::ssize()` are valid, they imply that we also need `std::ranges::ssize()` with the semantics of `std::ranges::size()` and a cast to a signed type.

The current state is inconsistent in regard to signed vs unsigned (inside `std::ranges::`).

Solution: add `std::ranges::ssize()`

3.2 `std::[s]size` should behave as `std::ranges::[s]size`

During discussion in LEWG it was criticised that we have different semantics in the different namespaces at all, i.e. that **the current state is inconsistent in regard to `std::` vs `std::ranges::`**. Because the types that `std::size()` applies to is a subset of the types that `std::ranges::size()` applies to, it was proposed to make them behave the same or even just have two functions instead of three or four.

It also became evident that this subsumption is only true now (before C++20), because, after C++20, ranges can opt-out of `std::ranges::size()` via `disable_sized_range` – but not out of `std::size()`. Thus any fix in this area must happen now.

The most obvious solution would be to have `std::size()` behave like `std::ranges::size()`, remove the latter, and have `std::ssize()` refer to `std::size()`. This is not possible, however, because `std::ranges::size()` is a function object and `std::size()` is a function that is subject to ADL. Replacing the function with a function object will break code that relied on ADL.

Solution: `std::size()` and `std::ssize()` are defined as functions that call their counterparts in `std::ranges::`. This guarantees that if we must have more than two interfaces, at least we only have two different semantics clearly denoted by name (signed vs unsigned).

4 Impact on the standard

See above why these changes would be breaking after C++20.

See below for wording.

5 Design decisions

This is the current proposal:

	`.size()` or `end() - begin()`
unsigned	`std::size()`, `std::ranges::size()`
signed	`std::ssize()`, `std::ranges::ssize()`

The discussion in LEWG also proposed the following other “solutions”:

1. **Replace `std::[s]size` with `std::ranges::[s]size`.** This doesn’t work because of function VS function object, see above.

2. **Only add `std::ranges::ssize()`, don’t touch `std::size()` and `std::ssize()`.** Less invasive, but only fixes the first inconsistency.

3. **Don’t add `std::ranges::ssize()` and remove `std::ssize()`.** This creates some consistency with regard to the absence of any signed size function. It doesn’t solve the second inconsistency above.

4. **Don’t add `std::ranges::ssize()`, but re-define `std::ssize()` in terms of `std::ranges::size()` instead of `std::size`.** It solves the problem of a lacking ssize for all ranges. However, it increases inconsistency within `std::`. And it does not solve the second inconsistency.

If there is no consensus to accept this proposal as a whole, I would still suggest doing option 2. Option3 would be possible, too, but is likely to decrease consensus in plenary.

6 Discussion (after R0)

6.0.1 Why not remove `std::ranges::size()` and give `std::[s]size()` the same semantics (but keeping them as functions and not making function objects)?

Possible, but ranges authors want size functions to be function objects. The change would also be quite invasive – because `std::ranges::size()` appears frequently. It’s too late to detect subtle breakage at this point.

6.0.2 `std::size(r)` is not a subset of `std::ranges::size(r)` right now, because the latter only considers `r.size()` if that returns something *integer-like* and the former has no such restrictions.

While I consider it very unlikely that code returns something not *integer-like* via `.size()` **and** also depends on exposing it via `std::size()` (especially since that was just introduced in C++17), changing this would break currently valid C++17 code. As such, I have changed the wording to maintain the current behaviour of `std::size()` and delegate to `std::ranges::size()` only for those arguments that are `sized_ranges` (which is most of the arguments likely used). `std::ssize()` is adapted in a similar way so that it can potentially work on types that are not `sized_ranges` but that `std::size()` works on.

This change means that adopting this paper after C++20 would still be possible and only "breaking" in the sense that `std::ssize()` on a `sized_range` could possibly return a different type in C++23 than it does in C++20.

6.0.3 `std::ranges::ssize()` does not work on those `sized_ranges` that have non-`std::integral` size type.

The reason is that `static_cast<common_type_t<ptrdiff_t, make_signed_t<decltype(E)>>>` is used to transform the type to signed type and this is not guaranteed to be valid for arbitrary *integer-like* types (that are allowed for `sized_ranges`; in particular `iota_view`). This shortcoming of R0 was known to the author and discussed briefly in LEWG. It was found that since this shortcoming is already part of the design of the original `std::ssize()` and can be fixed after C++20, it would not be addressed now.

Casey Carter expressed dissatisfaction with this approach and proposed various resolutions – one of which I have integrated in the current wording. Note that this affects the semantics of `std::ranges::ssize()` and `std::ssize()` for `sized_ranges`; but not `std::ssize`'s fallback solution described above.

7 Wording

§23.2 [iterator.synopsis]

```
-template<class C> constexpr auto size(const C& c) -> decltype(c.size());
-template<class T, size_t N> constexpr size_t size(const T (&array)[N])
  noexcept;
+template <class C> constexpr auto size(C&& c)
  noexcept(noexcept(c.size()))
+ -> decltype(c.size());
+template <ranges::sized_range C> constexpr auto size(C&& c)
+ noexcept(noexcept(ranges::size(forward<C>(c))))
+ -> decltype(ranges::size(forward<C>(c)));
-template<class C> constexpr auto ssize(const C& c)
+template<class C> constexpr auto ssize(C&& c)
+ noexcept(noexcept(c.size()))
   -> common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>;
+template <ranges::sized_range C> constexpr auto ssize(C&& c)
+ noexcept(noexcept(ranges::ssize(forward<C>(c))))
+ -> decltype(ranges::ssize(forward<C>(c)));
```

§23.7 [iterator.range]

```
-template<class C> constexpr auto size(const C& c) -> decltype(c.size());
-  Returns: c.size().
-
-template<class T, size_t N> constexpr size_t size(const T (&array)[N])
  noexcept;
-  Returns: N.
```

```
+template <class C> constexpr auto size(C&& c)
  noexcept(noexcept(c.size()))
+ -> decltype(c.size());
+  Returns: c.size().
+
+template <ranges::sized_range C> constexpr auto size(C&& c)
+ noexcept(noexcept(ranges::size(forward<C>(c))))
+ -> decltype(ranges::size(forward<C>(c)));
+  Returns: ranges::size(forward<C>(c)).

-template<class C> constexpr auto ssize(const C& c)
+template<class C> constexpr auto ssize(C&& c)
+ noexcept(noexcept(c.size()))
  -> common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>;
  Returns: static_cast<common_type_t<ptrdiff_t,
  make_signed_t<decltype(c.size())>>>(c.size())

-template<class T, ptrdiff_t N> constexpr ptrdiff_t ssize(const T (&array)
  [N]) noexcept;
-Returns: N.

+template <ranges::sized_range C> constexpr auto ssize(C&& c)
+ noexcept(noexcept(ranges::ssize(forward<C>(c))))
+ -> decltype(ranges::ssize(forward<C>(c)));
+  Returns: ranges::ssize(forward<C>(c)).
```

§24.2 [ranges.syn]

```
    inline constexpr unspecified size = unspecified;
+   inline constexpr unspecified ssize = unspecified;
    inline constexpr unspecified empty = unspecified;

    [...]

Within this clause, for some integer-like type X
  ([iterator.concept.winc]),
make-unsigned-like-t(X) denotes make_unsigned_t<X> if X is an integer
  type;
otherwise, it denotes a corresponding unspecified unsigned-integer-like
  type
of the same width as X.
For an object x of type X, make-unsigned-like(x) is x explicitly converted
  to
make-unsigned-like-t(X).
+ Within this clause, for some integer-like type X
  ([iterator.concept.winc]),
+make-signed-like-t(X) denotes make_signed_t<X> if X is an integer type;
+otherwise, it denotes a corresponding unspecified signed-integer-like
  type
+of the same width as X.
+For an object x of type X, make-signed-like(x) is x explicitly converted
  to
+make-signed-like-t(X).
```

$24.3 [range.access]

Introduce new §24.3.10 [range.prim.ssize] after §24.3.9 [range.prim.size]

```
+24.3.10 ranges::ssize
+
+The name ssize denotes a customization point
  object([customization.point.object]). The
+expression ranges::ssize(E) for some subexpression E is expression-
  equivalent to:
+make-signed-like(ranges::size(E)).
```